

Neural Networks and PyTorch

Understand how layers of trainable parameters learn through forward and backward passes.



WHY THIS MATTERS

Start from what you already know

Vectors, matrices, classification, loss functions, gradients, and learning rates

- 01 Explain tensors in plain language.
- 02 Use layers and activations in a small worked example.
- 03 Complete the guided lab and verify training and evaluation modes.

CORE CONCEPT

The vocabulary of this topic

Understand how layers of trainable parameters learn through forward and backward passes.

tensors: A tensor is a typed, multidimensional array that can live on CPU or accelerator memory and participate in differentiable operations.

layers and activations: A layer computes weighted sums; an activation introduces a nonlinear transformation.

backpropagation: Backpropagation applies the chain rule from loss to every trainable parameter.

training and evaluation modes: Training mode enables learning-time behavior; evaluation mode freezes stochastic or running-statistic behavior.

FOUR CONNECTED IDEAS

How the concept works

01

tensors

02

**layers and
activations**

03

backpropagation

04

**training and
evaluation modes**

$z=Wx+b$ computes a layer; an activation transforms z ; backpropagation applies the chain rule to calculate gradients.

CORE CONCEPT

Read every part of the mechanism

$z=Wx+b$ computes a layer; an activation transforms z ; backpropagation applies the chain rule to calculate gradients.

tensor: A typed multidimensional array.

weight: A trainable multiplier.

activation: A nonlinear transformation.

epoch: One pass over training data.

WORKED EXAMPLE

Worked example by hand

Calculate $z=2\times0.5-0.2=0.8$, apply ReLU to keep 0.8, then compare the output with its target.

01

Known values

02

Apply the mechanism

03

Interpret the result

RUN IT AND INSPECT THE RESULT

Map the concept to Python

```
import torch
x = torch.tensor([2.0])
w = torch.tensor([0.5], requires_grad=True)
loss = (torch.relu(x * w - 0.2) - 1.0) ** 2
loss.backward()
print(round(loss.item(), 2), round(w.grad.item(), 2))
```

OUTPUT

0.04 -0.8

Each line corresponds to a concept already explained.

QUALITY GATE

Build the complete mental model



Without nonlinear activations, stacked linear layers collapse into one linear transformation.



``loss.backward()`` calculates gradients; the optimizer uses them to update parameters.



Mini-batches trade noisy gradient estimates for memory and computation efficiency.



``model.eval()`` and inference mode make validation behavior stable and avoid gradient storage.

FOUR CONNECTED IDEAS

Notebook readiness map

01

tensors

02

**layers and
activations**

03

backpropagation

04

**training and
evaluation modes**

Use the concepts from this lesson to read and complete
<https://github.com/curiously/Al-Bootcamp/blob/master/02.real-world-pytorch.ipynb>.

RISK REVIEW

Common beginner mistakes

01 Forgetting to clear accumulated gradients.

02 Evaluating with dropout or batch normalization still in training mode.

GUIDED LAB

Train a small classifier with DataLoader batches and evaluate it under inference mode.

01

Predict

02

Run

03

Inspect

04

Explain

SESSION SYNTHESIS

Keep the system, not isolated definitions.

- 01 Explain tensors in plain language.
- 02 Use layers and activations in a small worked example.
- 03 Complete the guided lab and verify training and evaluation modes.

NEXT EVIDENCE

A PyTorch notebook showing shapes, forward pass, loss, gradients, optimizer updates, validation, and saved state.